

MA-SQLGrid: A Multi-Stage Context-Grounding Framework for Text-to-SQL over Power Grid Maintenance Databases

Bijing Liu ^{1,2}, Chenglong Sun ^{1,2} and Yong Yang ^{1,2,*}

¹ NARI Group Corporation (State Grid Electric Power Research Institute), Nanjing 211106, Jiangsu Province, China; author-email-required@example.com

² Beijing Kedong Electric Power Control System Co., Ltd., Beijing 100080, China; author-email-required@example.com

* Correspondence: yangyong1@sgepri.sgcc.com.cn

Featured Application

MA-SQLGrid provides a reproducible natural-language query layer for maintenance engineers who need auditable access to equipment, inspection, work-order, and outage records in a fixed relational database.

Abstract

Maintenance engineers increasingly need natural-language access to structured asset, inspection, work-order, and outage records, but text-to-SQL models remain sensitive to schema scale, domain literals, and answer-format requirements. MA-SQLGrid is a multi-stage context-grounding pipeline that selects schema and value evidence, normalizes power-grid expressions, infers the answer shape, generates SQL, and performs reference-free execution validation. On the held-out 180-question split of the synthetic GridDB-Maintenance-v2 benchmark, the compact contract-aware condition improves strict execution accuracy over full-schema/value prompting from 0.4389 to 0.7000 with one generator and from 0.3389 to 0.7000 with a second generator. Validation increases the corresponding scores to 0.7278 and 0.7667. Compact prompting reduces measured input tokens by 23.4% and 74.7% on the two serving stacks. Projection-tolerant rescoring reverses the compact-versus-full ordering, indicating that much of the strict gain arises from projection and ordering conformance rather than row-content retrieval. Three deterministic repeats show at most 1.1 percentage points of spread, and a tenfold row-scale diagnostic preserves the strict compact-context advantage. The results support the complete pipeline as an auditable interface for the tested synthetic maintenance database; cross-database and fully factorial mechanism claims remain outside the current evidence.

Keywords: Text-to-SQL; power grid maintenance databases; large language models; schema grounding; answer-shape inference; execution validation

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Appl. Sci.* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

1. Introduction

Text-to-SQL remains fragile in domains where the database is fully known yet still difficult to verbalize precisely. Power-grid maintenance is a representative case: questions refer to equipment status, inspection outcomes, work orders, outage history, and date-bounded conditions, but the SQL answer depends on exact schema links and literal values. A user may say “inactive transformer,” “latest inspection,” or “faulted feeder,” while the database stores those concepts under specific column names and canonical values. This

creates a recurring gap between natural language intent and executable SQL. The gap matters because the model can generate syntactically valid SQL that still returns the wrong denotation, especially when the question implies a particular answer shape such as a count, a grouped aggregate, a filtered list, or a scalar lookup. In a synthetic but deterministic maintenance database, the core challenge is therefore not open-ended reasoning. It is disciplined grounding against a fixed schema and value space [1–4].

Prior Text-to-SQL systems address pieces of this problem, but they tend to optimize one axis at the expense of another. Schema-linking methods improve token-to-column alignment, yet they still depend on the quality of the prompt context [2,5,6]. Prompt-based systems often help by adding demonstrations or structured cues, but full-schema prompting can inflate the context without solving literal mismatches [7–9]. Decomposition and self-correction methods improve reasoning over joins and filters, yet their gains depend on how intermediate steps are grounded [10–12]. Recent surveys also point to a stable pattern: the strongest systems are not simply larger, but more selective about which schema evidence they expose and how they verify outputs [13–15]. That observation motivates a narrower question for this benchmark: can compact domain context, not exhaustive schema exposure, improve a fixed generator on a frozen maintenance database?

MA-SQLGrid answers that question with a lightweight multi-stage design. A note on terminology: throughout this paper, MA-SQLGrid is described as a *multi-stage* prompting pipeline with fixed, role-specialized stages (schema selection, value normalization, answer-shape inference, generation, and validation). It is not an interacting multi-agent system: the stages execute in a fixed order, do not negotiate or debate, and share no learned coordination policy. We retain the system name MA-SQLGrid for continuity with the released artifact package, but we deliberately avoid the term “multi-agent” for the architecture itself. The framework first builds compact schema/value context, then canonicalizes power-grid values, then infers the expected answer shape, and only then generates SQL with one fixed non-frontier model. A final validator compares candidate SQL using execution behavior, answer-shape consistency, and value alignment without gold SQL. This structure follows the spirit of CHESS-style context harnessing [12] while remaining deliberately small enough to isolate the effect of grounding. The central idea is simple: in a narrow domain database, the generator should not see everything. It should see the few schema fragments and canonical values that most directly constrain the query, because those are the elements that determine whether the answer shape and the denotation are right [16–18].

The main contributions of this paper are summarized as follows:

- (1) It defines MA-SQLGrid, a compact multi-stage context-grounding framework for a deterministic power-grid maintenance database, with explicit schema selection, power-grid value normalization, and answer-shape inference in front of a single fixed generator.
- (2) It shows that the complete compact contract-aware condition outperforms schema-only and full-schema-values prompting under the same fixed generator (strict execution accuracy 0.7000 versus 0.4389), while explicitly separating this result from an unproven compactness-only mechanism because the study lacks a full 2×2 context-by-shape factorial.
- (3) It reports a reproducible evaluation protocol for GridDB-Maintenance-v2 v0.1, including the deterministic split, the evaluator contract, the comparator definitions, and an evaluation-convention sensitivity analysis that separates answer-contract conformance from row-content retrieval instead of taking the strict metric gap at face value [1,19,20].
- (4) It repeats the principal conditions with a second generator (deepseek-chat) under byte-identical prompts: the complete-pipeline gain (+36.1 points) and validation

increment (+6.7 points) have the same direction, the convention-sensitivity finding replicates, and three temperature-0 repeats bound accuracy spread at 1.1 points with 98.3% per-question verdict agreement. A deliberately bounded, asymmetric tenfold database-scale diagnostic records the associated accuracy and token behavior without claiming symmetric schema generalization.

Where the paper speaks of robustness, the word is used in a bounded sense: robustness to schema/value and answer-shape ambiguity inside the fixed GridDB-Maintenance-v2 v0.1 protocol. The paper makes no claim of cross-benchmark robustness, production readiness, schema-drift tolerance, or stability across multiple random seeds. This distinction is important because the study is designed to isolate the effect of context construction and validation on one controlled benchmark, not to evaluate broad deployment behavior.

The rest of this paper is organized as follows. Section 2 reviews prompting, decomposition, and validation methods and positions this work against them. Section 3 formalizes the problem and presents the MA-SQLGrid framework. Section 4 describes the dataset, comparators, metrics, and experimental protocol. Section 5 reports and discusses the test-split results, including the convention-sensitivity analysis and the second-generator validation. Section 6 concludes and states the limitations.

2. Related Work

2.1. Text-to-SQL Prompting and Schema Grounding

Prompt-based Text-to-SQL has shifted attention away from end-to-end training and toward how context is selected and structured [7–9]. This line of work is effective because a fixed generator can be steered with schema summaries, demonstrations, or explicit instructions, but the result is highly sensitive to what is included. If the prompt is too broad, irrelevant tables and values compete for attention. If it is too narrow, the model loses the evidence needed for joins or predicates. Prior work on schema linking and grounded adaptation shows that local alignment between the question and the schema is a major determinant of executable SQL [2,3,5,6]. MA-SQLGrid follows this direction, but it treats compactness as a first-class design goal rather than a byproduct of retrieval.

Content-aware prompting has also demonstrated that database values matter when the question names concrete literals rather than only schema entities [21–23]. That lesson is especially relevant in maintenance databases, where status labels and categorical values frequently decide whether a candidate query is correct. Exhaustive schema dumping, however, is often a poor tradeoff in a fixed, narrow database because it adds many irrelevant tokens without improving the few literal matches that matter. MA-SQLGrid differs by selecting a compact subset of schema and values and by canonicalizing those values into a stable domain vocabulary before generation.

2.2. Decomposition, Self-Correction, and Multi-Agent Orchestration

Decomposition-based methods improve Text-to-SQL by breaking a query into smaller decisions before generating the final SQL [10,24–26]. Multi-agent systems extend that idea by assigning distinct roles for decomposition, generation, and refinement [11,27,28]. These methods are attractive because they expose intermediate structure that can be checked against the question. Their drawback is overhead: each stage may reintroduce the same schema context, and the cost of multiple passes can outweigh the benefit when the database is simple and the schema is fixed.

MA-SQLGrid uses a similar role separation, but it is intentionally lighter. It does not rely on a large debate protocol or a broad search over many candidate generators. Instead, it uses compact schema/value context, answer-shape inference, and a single fixed generator, then validates the outputs with reference-free checks. This makes the method

closer to CHESS-style context harnessing than to a search-heavy agent pipeline [10–12]. The difference matters because this benchmark rewards careful grounding more than open-ended reasoning depth, so a smaller control loop is enough to expose the main effect.

Schema pruning, execution-guided selection, and error-driven repair are individually established in the systems above. MA-SQLGrid tests a narrower combination—domain-targeted value canonicalization and explicit answer-shape inference in front of one fixed generator—on a frozen operational database rather than claiming broad cross-domain superiority.

2.3. Execution-Based Evaluation and Validation

Execution accuracy remains the most meaningful metric for Text-to-SQL because exact string match is too brittle to capture semantic equivalence [1,14,29]. Execution-guided decoding and reranking use the database itself to reject malformed or inconsistent candidates [18,30,31]. More recent verification-oriented work extends this idea with semantic checks that do not require gold SQL at inference time [15,32,33]. MA-SQLGrid adopts that principle in its final selection stage, but limits the validator to lightweight, reference-free checks so that the method remains compact and contract-compatible.

This distinction is important. Validation can improve selection, but it also adds latency and can become brittle if it overweights shape or execution signals. MA-SQLGrid therefore treats validation as a refinement layer, not the main source of performance. That framing differentiates it from methods whose primary contribution is broader search, self-correction, or execution-aware reinforcement [11,34,35]. The present work focuses on a narrower claim: when context is already compact and domain-grounded, reference-free validation can improve the final choice without requiring gold SQL.

Recent Text-to-SQL work also reinforces the need to position compact domain grounding separately from broad benchmark generalization. Retrieval and schema-simplification systems such as CORE-T and UNJOIN focus on selecting coherent schema evidence before generation [36,37]. Multi-agent or refinement systems such as MARS-SQL, SEA-SQL, and SIRIUS-SQL emphasize iterative candidate improvement or execution feedback [38–40]. Evaluation-oriented work such as SpotIt+, SQLStructEval, and production SQL accuracy studies examine how to validate generated SQL beyond string match [41–43]. These very recent works are used as contemporary preprint context, not as settled archival baselines that define the claims of this paper. Closest to the present setting, Borovčák et al. [44] evaluated open-source LLM agents for SQL generation and structured analytics on relational databases in *Computers, Materials & Continua*, comparing model families and single-agent versus multi-agent architectures on business-analytics workloads and the Spider benchmark. MA-SQLGrid is complementary to that agent-evaluation perspective: instead of comparing generators and agent architectures, it fixes one generator and asks whether domain-grounded compact context construction and reference-free validation improve Text-to-SQL on a frozen operational power-grid database under an auditable local protocol. MA-SQLGrid intersects all of these themes, but its contribution is deliberately narrower: it tests whether a small, fixed-schema power-grid database benefits from compact schema/value grounding and reference-free validation.

3. Materials and Methods

3.1. Problem Formulation

Let a natural-language question be q , and let the database schema be S , with tables, columns, keys, and types extracted from `schema.sql`. Let V denote the available value hints drawn from the database contents and dataset annotations. The task is to generate

SQL y such that executing y on the database D yields the intended denotation a . We write the objective as

$$y^* = \arg \max_{y \in \mathcal{Y}} P(y \mid q, C(q, S, V)), \quad (1)$$

where $C(\cdot)$ is a compact context constructor that selects schema and value evidence relevant to the question.

MA-SQLGrid decomposes C into schema selection, value canonicalization, and answer-shape estimation. Schema selection identifies tables and columns likely to participate in the query. Value canonicalization maps domain expressions such as status labels, categories, and temporal forms to canonical database values. Answer-shape estimation predicts whether the query should return a scalar, grouped aggregate, set-valued list, or filtered record list. The generator then produces candidates conditioned on the compact context and the inferred shape. At inference time, no gold SQL is available, so selection must rely on the candidate SQL itself and the local database response.

3.2. Framework Overview

MA-SQLGrid is designed around a narrow but useful objective: for a question x , a fixed schema S , and a database instance D , produce SQL that executes to the intended denotation. The system does not learn a new parser. Instead, it changes how the input to a fixed generator is assembled and how candidate outputs are chosen. That choice matches the structure of the benchmark. In a frozen maintenance database, many errors come from overexposed schema context, literal mismatch, or an incorrect answer shape, rather than from a lack of general language competence [10,12,16].

Formally, the framework constructs a compact context c from x , S , and V using three components: a selector g , a normalizer n , and a shape predictor h . The resulting context is

$$c = g(x, S, V) \oplus n(x, V) \oplus h(x), \quad (2)$$

where $\oplus$ denotes concatenation into a prompt-ready representation. Given that context, the fixed generator produces a small set of candidate SQL strings,

$$\{y_1, \dots, y_k\} \sim p_\theta(y \mid x, c), \quad (3)$$

and the validator assigns each candidate a score that combines execution success, answer-shape consistency, value alignment, and a small complexity penalty:

$$s(y_i; x, c, D) = \lambda_1 \text{Exec}(y_i, D) + \lambda_2 \text{Shape}(y_i, h(x)) + \lambda_3 \text{Value}(y_i, n(x, V)) - \lambda_4 \text{Cost}(y_i). \quad (4)$$

The final prediction is the highest-scoring executable candidate. This objective is reference-free at inference time and depends only on the candidate SQL, the compact context, and the database behavior [1,15,32]. Fig. 1 summarizes the architecture, and the rest of the implementation follows a fixed schedule: question analysis, domain normalization, compact prompt assembly, generation, execution, and selection. The method is therefore best understood as a controlled multi-stage prompting system rather than a search-heavy agent loop. It is lightweight by design, but each stage removes a specific failure mode that the benchmark makes visible.

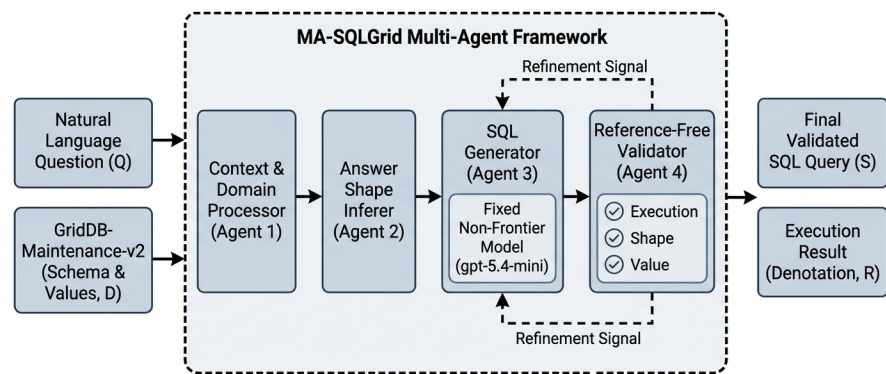


Figure 1. MA-SQLGrid architecture: fixed multi-stage schedule from question analysis through compact context construction, generation, and reference-free validation.

3.3. Compact Schema/Value Selection

The first component is compact schema/value selection. The component ablation of Section 5 attributes the largest single share of the end-to-end gain to the answer-shape channel that this compact context carries (removing shape hints drops correct answers from 126 to 77), with the schema/value reduction itself contributing through disambiguation and attention economy rather than dominating the gain on its own. MA-SQLGrid does not expose the full schema. It scores schema items by lexical overlap with the question, relation proximity, and domain salience, then keeps only the subset needed to express the likely joins and predicates. In the implementation used for the formal run, lexical matching is applied to table names, column names, and value mentions; selected tables are expanded through foreign-key neighbors up to a fixed cap; and the domain selector prioritizes maintenance-relevant entities such as assets, work orders, sensor readings, and topology edges when the question contains corresponding operational phrases. The same reduction applies to values: only the hints that help disambiguate maintenance categories, status labels, and temporal forms are retained. This is not a compression trick for its own sake. In a narrow database, irrelevant schema tokens do not improve coverage; they dilute attention. The test results support that choice: the compact-domain condition reaches 0.7 execution accuracy while using far fewer estimated prompt tokens than the full-schema baseline. That pattern is consistent with selective grounding work in schema linking and structured retrieval [2,4,17,45].

3.4. Power-Grid Value Normalization

The second component, power-grid value normalization, maps question phrasing into canonical database literals before generation. The normalizer is a fixed domain dictionary and pattern set over the local value inventory, not a learned classifier and not a lookup into gold SQL. If the question uses an abbreviated or colloquial form, the normalizer resolves it to the local database value that the SQL must contain. If the question mentions a status or asset class, the system converts that mention into the dataset's canonical vocabulary. For example, phrases such as "offline assets," "closed topology connection," and "temperature alarm" are converted into candidate predicates over the corresponding status, topology, or sensor columns. The local inventory used by the formal run contains 196 distinct values across 28 non-ID categorical, date, status, flag, and text columns; on the 180-question test split, the domain context emitted at least one normalized value hint for 170 questions, with 330 hint instances and 111 unique rendered hints. This matters because the benchmark is deterministic: value mismatches are not random noise but predictable lexical failures. Separating normalization from generation makes the generator's task smaller and more stable. It also explains why MA-SQLGrid improves on schema-only and full-schema-values

prompting: the prompt is not merely shorter or longer, it is aligned with the database's literal conventions [3,22,46].

3.5. Answer-Shape Inference

The third component, answer-shape inference, predicts the form of the answer before SQL is generated. Let $\mathcal{T}$ be a small set of shape classes such as scalar, grouped aggregate, set-valued list, filtered record list, and boolean. The predictor computes

$$h(x) = \arg \max_{t \in \mathcal{T}} p_{\phi}(t \mid x), \quad (5)$$

where ϕ is a prompt-derived policy inside the framework. The purpose is to constrain the candidate space, not to solve semantics independently. Questions that ask “how many,” “which,” “whether,” or “latest” tend to require different SQL skeletons, and the wrong skeleton often yields a valid but semantically wrong query. Injecting shape into the context gives the generator an inexpensive structural prior. That is why the generic CHESS-style baseline improves shape accuracy relative to schema-only prompting, but the domain-grounded version improves much more once shape is coupled with canonical value hints [11,12,24].

3.6. Generation, Reference-Free Validation, and Bounded Repair

The generator stage uses one fixed non-frontier model, gpt-5.4-mini, with deterministic decoding at temperature 0; a second-generator replication with deepseek-chat under byte-identical prompts is described in Section 4. Fixing the model isolates the framework effect from any effect of a stronger generator family. The model receives the compact schema/value context, the normalized domain hints, and the answer-shape cue, then returns candidate SQL in a controlled format. Deterministic decoding is useful here because it turns the comparison into a question of context and validation, not sampling variance. Exact future reproducibility can still be affected by hosted-model drift, so the artifact package retains the model identifier, prompts, context hashes, raw outputs, selected SQL, retries, token estimates, and serving logs needed to audit the reported run [7–9].

The final stage is reference-free execution, shape, and value validation. Each candidate is executed on the local SQLite database. Candidates that fail syntax or execution are discarded. Among executable candidates, the validator checks whether the observed result shape matches the inferred answer shape and whether the SQL predicates align with the normalized value hints. The selected query is the one with the highest composite score. The cost is linear in the number of candidates, so the validation layer increases latency, but it also explains the measured improvement in answer-shape accuracy and final execution accuracy [15,18,30].

The ranking policy is conservative by design. Executable candidates outrank non-executable ones; among executable candidates, shape-consistent queries outrank shape-inconsistent ones; and among those, candidates with stronger value alignment and lower syntactic complexity are preferred. The implementation uses fixed integer weights for these checks: safety and execution success receive the largest positive weights, shape agreement and ordering agreement receive smaller positive weights, empty results and missing value hints are penalized, matched value hints add reward, and candidate order is used as a deterministic tie-breaker. These weights are fixed before reporting and are not tuned on the test split. The system never requires gold SQL, gold answers, or hand-coded rules for specific labels. Its signal comes from the candidate SQL and the database response. That aligns the validator with the same execution contract used in the experiments, which is essential for a paper focused on validated denotation rather than string matching.

For reproducibility, the C5 condition fixes the candidate and repair policy rather than leaving it to an open-ended agent loop. C5 generates at most five candidate SQL strings from the same compact domain context used by C4. Each candidate is checked by the local read-only SQL guard before execution. The ranking signal combines four reference-free checks: whether execution succeeds, whether the observed result shape is compatible with the inferred answer shape, whether the SQL preserves the inferred value hints, and whether ordering cues are respected when the question implies an ordered or top-k answer. If two candidates receive the same effective validation status, the earlier generated candidate is preferred, which gives deterministic tie-breaking under temperature-zero decoding.

The repair boundary is also fixed. C5 permits one repair opportunity when the initially selected candidate fails execution, violates the inferred shape, misses inferred value hints, or ignores an inferred ordering cue. The repair prompt receives the original question, the compact context, the failed SQL, and the reference-free validation signal. It does not receive gold SQL, gold denotation rows, evaluator correctness, required-literal metadata, answer-shape metadata, or order-sensitive metadata. A repaired SQL string is accepted only if it executes and does not worsen the available reference-free validation signal. If no candidate can be extracted, the runner records an error and emits a safe placeholder rather than silently fabricating a successful query. The archived formal run reported zero contract errors, zero unsafe SQL records, zero provider/extraction errors, and zero leakage-scan problems. Thus, the validation layer is reproducible as a bounded selector over generated candidates, not as an unconstrained self-correction process. The full set of implementation details fixed in the reported run, including the schema-selection rules, the C5 ranker weights, and the no-gold prediction contract, is provided in Table S1 of the Supplementary Materials.

4. Experimental Design

4.1. Dataset and Database

The evaluation uses only the verified local dataset GridDB-Maintenance-v2 v0.1, with the schema, SQLite database, question file, split file, annotation protocol, and verification log retained in the artifact package. It contains 200 question-SQL pairs and follows a deterministic split: Q001–Q020 are dev and Q021–Q200 are test. The dev split is reserved for feasibility checks and prompt calibration, while the formal comparison uses the held-out 180-question test split. That split discipline is central to the paper because the dataset is small, frozen, and deterministic, so leakage would be easy to miss but would distort every downstream claim.

The database contains eight tables covering asset types, assets, grid topology, locations, maintenance logs, sensor readings, technicians, and work orders. The schema is compact but relationally connected. The largest entity table is assets with 18 rows and seven columns for asset identity, type, location, installation date, status, and capacity. The work-order table contains 15 rows and covers priorities, statuses, scheduled and completed dates, assigned technicians, and fault codes. The sensor table contains 26 readings with sensor type, value, unit, timestamp, and alarm flag. Supporting tables provide six asset types, eight locations, eight technicians, eight maintenance logs, and nine topology edges. This structure creates a useful test bed for schema/value grounding because many questions require joins from assets to asset types, locations, work orders, readings, or topology relations.

The value space is small enough to enumerate but rich enough to create literal-binding failures. Asset types are Breaker, Capacitor, Line, Relay, Substation, and Transformer; voltage classes are 33kV, 115kV, and 230kV; asset statuses are *in_service*, *maintenance*, and *offline*; location regions are Central, East, North, South, and West; work-order

statuses are completed, open, and scheduled; priorities are high, medium, and low; and fault codes include BREAKER, CAP, INSPECT, LINE, LOAD, RELAY, SUB, TEMP, and VOLT. Other important vocabularies include technician specialties, sensor types (load, power_factor, temperature, voltage), sensor units (C, MW, kV, pu), topology connection types (control, feeder, tie), and switch states (closed, open). These values explain why exhaustive schema exposure is not enough: the correct SQL often depends on mapping a natural-language phrase to one of a few exact local literals.

The question annotations also show why answer-shape control is central. Among the 200 questions, 187 include a filter tag, 131 include an order-by tag, 113 require a join, 66 involve aggregation, 33 include a time predicate, 20 involve top-k selection, 16 use group-by, 12 involve topology traversal, and 10 require a self-join. The difficulty labels are 74 easy, 102 medium, and 24 hard. The most frequently referenced table is assets with 120 question annotations, followed by work orders with 72, locations with 54, asset types with 40, sensor readings with 31, technicians with 19, grid topology with 12, and maintenance logs with 7. This distribution makes the benchmark a focused test of join selection, literal normalization, ordering, and result-shape control rather than a generic SQL stress test. A tabular dataset characterization is provided in Table S2 of the Supplementary Materials.

4.2. Conditions and Generators

The formal comparison centers on five short condition labels, C1 through C5, chosen to isolate the effect of compact grounding and validation. C1 is a schema-only direct prompt. C2 exposes the full schema and values in a direct prompt. C3 is a CHERS-inspired generic multi-stage prompt without power-grid-specific normalization and shape discipline; it is a prompt baseline motivated by CHERS, not a reproduction of the complete CHERS system. C4 is the proposed compact domain-grounded method. C5 adds reference-free validation and candidate ranking. The verbatim templates for all five conditions are reproduced in Section S1 of the Supplementary Materials.

The main generator is a fixed gpt-5.4-mini configuration with temperature=0. Fixing the model matters because it removes generator-family variation from the comparison. The paper is therefore not a study of model scaling. It is a study of how context construction and validation change the behavior of one fixed generator on a narrow deterministic database. The exact serving logs, prompts, context hashes, model identifier, retries, token estimates, predictions, and scores are retained with the artifact package for reproducibility.

To test whether the reported effects are specific to one generator family, the study adds a second-generator protocol. The C2, C4, and C5 conditions were re-run with deepseek-chat, accessed through the provider's direct OpenAI-compatible endpoint (api.deepseek.com/v1) at temperature 0, on the same 180 held-out test questions. The second-model runner reuses the archived formal-run prompts byte-for-byte, so both generators receive identical inputs for C2 and C4 and identical candidate-generation prompts for C5. The C5 validation, ranking, and repair stage, whose original implementation is tied to the archived pipeline, was reimplemented from the specification in Section 3 (identical guard, ranker weights, and bounded-repair policy) and validated against the archive before use: on the archived gpt-5.4-mini candidate sets it reproduces 167 of 169 non-repair selection decisions (98.8% agreement). Scoring uses the same packaged evaluator under the same strict contract. The second-generator run comprised 540 calls with zero provider errors and a safe-SQL rate of 1.0 in all three conditions. A companion consistency check repeated C4 and C5 three times each at temperature 0 on the same generator (1,080 additional calls) to bound run-to-run variation. Both are reported in Section 5.4.

The intended release package contains the dataset files, schema, SQLite database, split file, annotation protocol, evaluator, condition prompts, context hashes, prediction and score

records, trace files, component-ablation addendum outputs, validator diagnostics, final $\text{\LaTeX}$ /PDF sources, and citation-verification artifacts. This release boundary is important because the paper’s claims are local to the fixed artifact package rather than to a mutable hosted endpoint or an unreported private benchmark.

Each test question is converted into a condition-specific prompt, sent to the fixed generator, optionally passed through reference-free validation, executed against `database.sqlite`, and scored by the same local semantic evaluator. Only context construction and validation differ across the ablations.

4.3. Metrics, Prediction Contract, and Evaluation Conventions

The primary metric is execution accuracy by validated denotation, computed with the local semantic evaluator in `evaluator/evaluator.py`. If N is the number of test questions and $z_i \in \{0, 1\}$ indicates whether the predicted SQL for question i matches the validated denotation, then execution accuracy is

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N z_i. \quad (6)$$

Higher is better, and the value lies in $[0, 1]$. Secondary metrics include safe-SQL rate, executable-SQL rate, answer-shape accuracy, prompt token estimate, latency, provider failure rate, and unsafe-SQL rate. Safe-SQL rate measures whether the prediction passes the read-only SQL guard. Executable-SQL rate measures whether the guarded query actually executes on `database.sqlite`. These two rates are reported separately because a query can be safe and still fail at execution time, for example by using a nonexistent column. Answer-shape accuracy measures agreement between the predicted result shape and the inferred or annotated shape. Prompt token estimate tracks input size. Latency measures end-to-end runtime per question. Provider failure rate and unsafe-SQL rate are reported because they distinguish model instability from benchmark difficulty.

All SQL generation ran through the hosted model API; the local machine only assembles prompts, executes SQLite queries, and scores results, so no local GPU is involved in any reported number. All five conditions achieved a safe-SQL rate of 1.0, with no provider failures and no unsafe SQL. Executable-SQL rates were 1.0000 for C1 and C2, 0.9167 for C3, 0.8889 for C4, and 0.9722 for C5. These numbers matter because they separate safety and availability from semantic correctness. The main variation is therefore whether the framework selects an executable query with the right denotation and answer shape.

The experiment uses a fixed prediction contract so that the comparison is auditable. Each record stores the condition, prompt/context hashes, selected SQL, candidate list, selected-candidate index, trace path, latency, token estimates, retry count, and error field. It excludes gold SQL, gold result rows, evaluator correctness labels, expected result hashes, required-literal metadata, answer-shape metadata, and order-sensitive metadata. Those excluded fields are used only after prediction generation for scoring and diagnostics. Likewise, the held-out audits, residual-error tables, and illustrative examples in this paper are post-generation diagnostics; they did not feed the prompts, normalizer rules, ranker weights, repair prompts, or candidate-selection decisions. This boundary is essential because C4 and C5 depend on inferred shape and inferred value hints; using dataset metadata in prompts or rankers would no longer test the proposed mechanism. The formal run produced 900 predictions and 900 score records, corresponding to 180 test questions times five conditions under the deterministic seed label, with zero contract errors, zero unsafe SQL records, zero provider or extraction errors, and zero leakage-scan problems.

Two evaluation conventions inherited from the dataset annotation protocol must be stated explicitly, because they interact with the proposed method. First, every multi-

row gold query carries a deterministic ORDER BY clause, and 131 of the 200 questions are annotated order-sensitive, so the strict evaluator compares row sequences for those questions. Second, every gold query projects exactly the minimal column set recorded in the annotation’s answer-shape metadata, and the strict evaluator rejects any prediction whose column count differs. The answer-shape inference stage of MA-SQLGrid encodes precisely these two conventions as question-derived hints (expected projection count, row granularity, and a deterministic-ordering flag). All five conditions receive the same instruction text asking the model to match the requested projection and use deterministic ordering, but only C4 and C5 receive the inferred hints that make the conventions explicit for the specific question; C1–C3 must recover them from the question wording alone. Part of the headline gap between C4/C5 and the baselines is therefore attributable to convention conformance rather than to row-content grounding. Section 5.2 quantifies this dependence directly by reporting two auxiliary relaxed metrics alongside the strict metric, so the reader can separate the two effects instead of taking the strict gap at face value.

The single-pass protocol should be interpreted carefully. The formal seed value is a reproducibility label for the deterministic evaluation, not a random-seed sample from which run-level variance can be estimated. Two pieces of trace-level determinism evidence bound the protocol risk: decoding temperature was fixed at 0 for every call, and of the 900 archived API calls, 898 succeeded on the first attempt while 2 (one in C1, one in C4) required a single retry; the final records contain no provider errors. This documents that the reported single pass was not silently stabilized by repeated sampling, but it does not replace a repeated-run consistency study; such a study is provided for the second generator in Section 5.4. The paired sign tests compare per-question binary outcomes within the fixed 180-item test set, so they are useful for showing how often one condition answers a question correctly when another condition does not. They do not establish multi-run stability.

5. Results

5.1. Main Results

Table 1 reports the formal protocol summary on the held-out test split.

Table 1. Formal protocol summary for the held-out GridDB-Maintenance-v2 v0.1 test split (180 questions, fixed gpt-5.4-mini generator, temperature 0). Best execution accuracy in bold.

Condition	Role	Prompt tokens	Latency ms	Executable SQL	Execution accuracy
C1	Schema-only direct generation	381.3	3259.8	1.0000	0.3944
C2	Full schema and values direct generation	710.3	2959.0	1.0000	0.4389
C3	CHESS-inspired prompt baseline	202.0	2895.1	0.9167	0.4000
C4	Compact domain context	259.2	2887.5	0.8889	0.7000
C5	Compact context with validation	258.2	5562.5	0.9722	0.7278

Table 1 captures the core resource tradeoff. The estimated prompt-token column measures the condition-specific context template rendered by the experiment runner: the full-schema context is almost three times the size of the compact-domain context (710.3 versus 259.2 estimated tokens, a 63.5% reduction), yet it performs substantially worse. The validation condition keeps the prompt nearly as small as the unvalidated compact variant, but its latency rises because it scores multiple candidates. The gain is not free, but it is also not purchased by simply exposing more context.

Table 2. Measured API resource consumption per question, aggregated from the 900 archived provider-reported usage records (180 questions per condition). Input/output tokens are the provider’s counts, which include serving-side overhead shared by all conditions; C5 combines candidate generation and, when triggered, one repair call.

Condition	Input tokens (mean)	Output tokens (mean)	Latency ms (mean)	Latency ms (median)	First-attempt calls
C1	5007.7	45.0	3259.8	1998.0	179/180
C2	6346.7	45.8	2959.0	1999.5	180/180
C3	4756.3	47.3	2895.1	1959.0	180/180
C4	4859.0	52.0	2887.5	2102.5	179/180
C5	5309.8	197.5	5562.5	3845.5	180/180

Table 2 complements the estimated column with the provider-reported token usage of the actual run, and the two views must not be conflated. The measured input cost of C4 is 4859.0 tokens per question against 6346.7 for C2, a 23.4% reduction, far smaller than the 63.5% template-level reduction. The difference is a fixed serving-side overhead of roughly 4500–4600 input tokens per call that is identical across conditions (visible in C1, whose template is only 381.3 estimated tokens but whose measured input is 5007.7). The honest efficiency statement is therefore: compact grounding shrinks the controllable part of the prompt by about a factor of 2.7, which translated into a 23.4% end-to-end input-token saving under this serving stack; deployments with less fixed overhead would realize proportionally more. The second-generator run (Section 5.4) confirms this reading on a clean endpoint: on deepseek-chat, served directly by its provider with negligible fixed overhead, the same C4-versus-C2 contrast yields a measured 74.7% input-token reduction (509.3 versus 2011.6 tokens per question). The three numbers triangulate consistently: the template-level estimate (63.5%) sits between the overhead-masked measurement on the original stack (23.4%) and the near-overhead-free direct measurement (74.7%), so the direct endpoint corroborates, rather than contradicts, the template estimate. On the output side, C5 consumes about four times the completion tokens of C4 (197.5 versus 52.0 per question) because it requests multiple candidates and occasionally a repair, which is the direct cost of the validation layer. The complete formal run consumed 4.73M input tokens and 69.8k output tokens across the 900 calls.

The formal results were obtained in one deterministic pass over the test split, so the reported values are single-run outcomes rather than averages over repeated seeds. The paired comparisons (Supplementary Table S3) show that C4 is correct on 49 questions that C2 misses, while the reverse holds for only 2 questions (mean difference +0.2611, exact sign test $p = 1.179 \times 10^{-12}$); C5 versus C2 shows 56 versus 4 such questions (+0.2889, $p = 9.085 \times 10^{-13}$). By contrast, C5 versus C4 is 12 versus 7 (+0.0278, $p = 0.3593$). Thus, the compact condition changes strict contract-compliant success reliably, whereas the additional validation layer provides only a nonsignificant numerical increment in the original run and is not claimed as a confirmed standalone gain.

Fig. 2 visualizes the execution-accuracy gap across conditions. C4 captures the principal grounding gain without added latency, whereas C5 incurs multi-candidate validation cost for a smaller additional gain. The Wilson intervals are item-level intervals over test questions, not confidence intervals over repeated model runs.

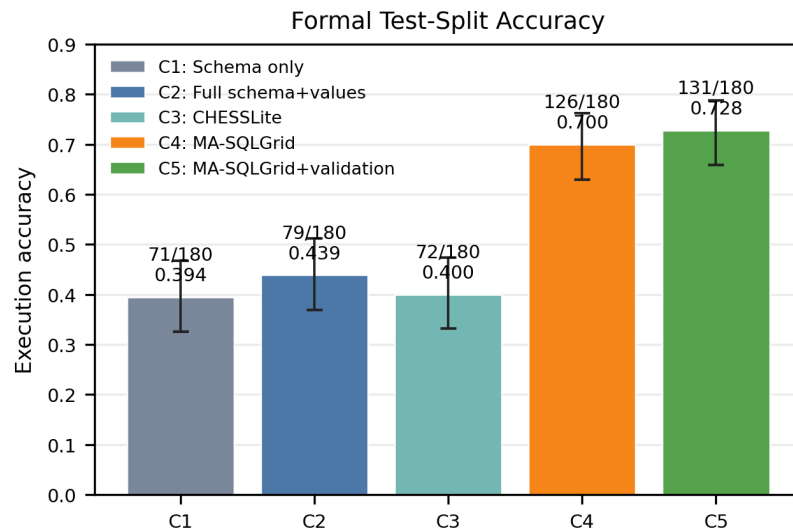


Figure 2. Formal test-split execution accuracy by condition with item-level Wilson intervals over the 180 test questions.

The answer-shape metric adds a second layer of interpretation. C1 reaches answer-shape accuracy of 0.3278, C2 reaches 0.3667, C3 reaches 0.5056, C4 reaches 0.8889, and C5 reaches 0.9722. The shape gain from C3 to C4 is substantial, which suggests that generic multi-stage prompting is not enough on its own. Shape inference matters most when it is paired with domain normalization. The validated condition then improves shape consistency again, which indicates that validation is not only rejecting invalid SQL but also preferring candidates whose result form matches the question.

Value-hint coverage is treated as an internal trace diagnostic rather than a headline metric: in the runner, a prediction with no inferred value hints receives a vacuous coverage value of 1.0, so this quantity is useful for auditing domain-context traces but not for ranking conditions. The stronger evidence for value grounding is instead the paired accuracy gap between C4 and C2. The trace-level coverage audit helps interpret this point without changing the main metric: among the 180 held-out test questions, 170 produced at least one normalized value hint in the domain context, with 330 normalized hint instances and 111 unique rendered hints over 20 matched value columns. These numbers are not used to score predictions, but they show that the normalizer was active on most of the formal split rather than only on a small hand-picked subset.

5.2. Evaluation-Convention Sensitivity

All 900 predictions were also re-scored with order-insensitive and projection-tolerant variants. Relaxing order alone preserves the condition ranking, but projection tolerance raises C2 from 0.4389 to 0.8722, above C4 at 0.7444; C4 itself gains nothing from projection tolerance. Thus, the large strict advantage is dominated by answer-contract conformance—minimal projection and deterministic ordering—rather than superior row-content retrieval. The strict metric remains operationally relevant, but the relaxed result bounds the claim: compact grounding improves contract-compliant answers on this benchmark, not the baseline’s ability to locate rows. Table 3 reports the corresponding strict and relaxed values for both generators and shows that the reversal replicates on deepseek-chat.

5.3. Component Ablation

To separate the two prompt-side components more directly, an addendum run removed one C4 hint channel at a time while preserving the same held-out test split, generator,

temperature-zero decoding, and no-gold prediction contract. The addendum does not replace the formal C1–C5 comparison, but it clarifies the mechanism. Removing value hints reduces C4 from 126/180 to 118/180 correct. Removing answer-shape hints reduces it to 77/180 correct and lowers answer-shape accuracy to 0.4389. The result supports a bounded component-level interpretation: value normalization contributes a measurable additional gain, while answer-shape hints are a central part of the compact-context effect on this benchmark.

5.4. Second-Generator Validation and Consistency

The second-generator protocol described in Section 4 re-ran C2, C4, and C5 with deepseek-chat on the same 180 held-out test questions using byte-identical archived prompts and the same packaged evaluator. Table 3 reports strict and relaxed metrics for both generators side by side.

Table 3. Cross-model comparison on the 180-question test split under strict and relaxed metrics. The deepseek-chat rows use byte-identical prompts, temperature 0, and the same evaluator.

Generator	Condition	Strict (primary)	Order-insensitive	Order-insensitive + projection-tolerant
gpt-5.4-mini	C2	0.4389 (79/180)	0.4778	0.8722
gpt-5.4-mini	C4	0.7000 (126/180)	0.7444	0.7444
gpt-5.4-mini	C5	0.7278 (131/180)	0.7722	0.8056
deepseek-chat	C2	0.3389 (61/180)	0.3667	0.8500
deepseek-chat	C4	0.7000 (126/180)	0.7944	0.7944
deepseek-chat	C5	0.7667 (138/180)	0.8278	0.8278

Three findings replicate. First, the compact-context gain replicates and is larger on the second generator: C4 improves over C2 by +36.1 points strict execution accuracy (0.3389 to 0.7000) on deepseek-chat, versus +26.1 points (0.4389 to 0.7000) on gpt-5.4-mini. The C4 strict score is coincidentally identical on both generators (126/180). Second, the validation-layer gain replicates and is also larger: C5 adds +6.7 points over C4 (0.7000 to 0.7667) versus +2.8 points on the original generator. Third, and most importantly for the interpretation in Section 5.2, the projection-tolerant reversal replicates: under the fully projection-tolerant reading, C2 exceeds C4 on both generators (0.8500 versus 0.7944 on deepseek-chat; 0.8722 versus 0.7444 on gpt-5.4-mini), while the compact conditions gain nothing from projection tolerance (C4 and C5 are unchanged on deepseek-chat). The conclusion that the strict gain is dominated by answer-contract conformance rather than row-content retrieval is therefore a cross-model result, not a single-model observation.

The resource profile of the second-generator run (Table S4 in the Supplementary Materials) also replicates the efficiency findings on a clean endpoint. Because the direct endpoint adds almost no fixed per-call overhead, the measured input tokens track the rendered prompts closely: C4 consumes 509.3 input tokens per question against 2011.6 for C2, a 74.7% measured reduction, consistent in magnitude with the 63.5% template-level estimate, whereas the original serving stack showed only 23.4% because of its roughly 4,500–4,600-token fixed overhead per call (Table 2). The output-side pattern of the original run also replicates: C5 consumes about four times the completion tokens of C4 (186.2 versus 46.6) and roughly triples mean latency (3014.2 versus 1054.0 ms), which is the direct cost of multi-candidate validation. C2 and C4 completed all 360 calls on the first attempt; C5 required a single retry on 2 of 180 questions. The full three-condition run consumed 576.3k input and 50.9k output tokens over 540 calls, with zero provider errors.

The companion consistency check bounds run-to-run variation directly: C4 and C5 were each run three times at temperature 0 (1,080 calls total). Per-repeat strict execution accuracy was 0.7056/0.6944/0.7000 for C4 and 0.7667/0.7667/0.7611 for C5, a maximum spread of 1.1 points. Exact SQL string agreement across all three repeats was lower (82.2%

of questions for C4, 77.2% for C5), confirming that token-level sampling nondeterminism exists on hosted endpoints even at temperature 0; however, all three repeats produced the same correct/incorrect verdict on 98.3% of questions in both conditions. Combined with the trace-level evidence from the original run (898/900 first-attempt calls at temperature 0), the aggregate metrics and per-question verdicts reported in this paper are highly stable under repetition, even though individual SQL strings are not guaranteed to be byte-identical across runs.

The boundary of this validation should be stated as precisely as the result. The study now covers two generator families, one closed (gpt-5.4-mini) and one open-weight served through its provider endpoint (deepseek-chat); no third family was tested, and the cross-model comparison is at the aggregate level, with no per-question significance test across models. The C5 stage for the second generator is a specification-faithful reimplementation validated at 98.8% selection agreement against the archive, not the original binary. Within those bounds, every headline effect of the paper replicates with the same sign and larger magnitude.

5.5. Scale Robustness at Tenfold Database Size

A deterministic tenfold row-scaling stress test (180 assets plus two distractor tables) was evaluated with deepseek-chat on the same 180 questions. C4 reaches 0.5944 versus 0.3333 for C2, and C5 reaches 0.6500; C2 input grows by 51% while C4 stays near 504 tokens per question. This is a bounded engineering diagnostic, not a symmetric schema-generalization test: the distractor tables enter the full-schema prompt but not the compact builder's fixed eight-table catalog. The test nevertheless exposes a genuine LIMIT 80 value-inventory failure, so deployment requires a scalable value index and a future stress test in which semantically confusable tables enter both candidate paths [12]. Full metrics are reported in Supplementary Section S4.

5.6. Error Analysis and Case Studies

The archived taxonomy (Supplementary Table S6) shows that shape mismatch dominates C1–C3 but is no longer the primary residual failure for C4/C5. C4 has 34 wrong-denotation and 20 execution errors; C5 reduces execution errors to five while retaining 44 wrong-denotation errors, confirming that validation repairs executability but cannot use gold denotations. The remaining failures are schedule-column naming errors rooted in a schema-selection miss. Full trace diagnostics, tag-level results, the claim-to-evidence map, and the three SQL case studies are provided in Supplementary Tables S5–S7 and Section S2.

6. Discussion

The main result is that compact, domain-normalized context is more effective than exhaustive schema exposure for this database. That pattern is consistent with prior schema-linking and grounded prompting work, but the present benchmark makes the mechanism unusually clear [2–4,17]. When the schema is frozen, the key question is not how much evidence to include. It is which evidence actually constrains the join path, predicate value, and answer shape. MA-SQLGrid answers that question by selecting a smaller but better aligned prompt, and the execution gain indicates that the generator benefited from that reduction. A second finding is that answer-shape control matters more when it is coupled to value normalization. The generic CHESS-style baseline improves shape accuracy relative to schema-only prompting, but it does not reach the same execution level as MA-SQLGrid because it lacks the domain-specific canonicalization step [10–12]. The component ablation makes the same point from the opposite direction: removing shape hints causes a large accuracy drop, while removing value hints causes a smaller but still measurable drop. A

query can have the right structure and still return the wrong rows if the status or category literal is off by one canonical form; conversely, literal hints are much less useful when the generated SQL has the wrong projection or ordering form.

The validation layer deserves a separate interpretation. Its gain over compact grounding is smaller than the gain from compact grounding over full-schema prompting, which is exactly the pattern a reference-free selector should produce [15,18,30]. Validation can clean up remaining shape and value inconsistencies, but it cannot replace good context construction. That is why the validated variant improves answer-shape accuracy most visibly and execution accuracy more modestly. The method therefore behaves as a two-stage system: grounding does the heavy lifting, and validation corrects a subset of residual errors.

The runtime tradeoff is meaningful for deployment. The compact-domain condition is not slower than the full-schema baseline, so the improvement is not bought with a wider prompt; the validated condition is slower because it evaluates multiple candidates. If the task is interactive and latency-sensitive, C4 is the preferred condition because it captures the main gain without the validation overhead. If correctness outweighs latency, C5 is the stronger choice. Prior agentic and execution-guided systems have often emphasized broader search or heavier refinement loops [11,35], but this benchmark suggests that a smaller control loop can be enough when the schema is fixed and the domain vocabulary is narrow. The paper therefore does not present validation as universally superior: C5 is best overall, but the most important scientific result is the C4 jump over the non-MA-SQLGrid baselines.

The broader implication is modest but useful. In this tightly scoped database, the compact contract-aware prompt outperforms a fuller prompt under the strict evaluator. This is relevant to power-grid maintenance tasks, where literal matching and answer-shape control are important, but it should not be read as a general rule that shorter prompts are intrinsically better. A validator over poorly grounded candidates cannot reliably recover missing domain evidence, while a grounded candidate set gives the selector more useful alternatives. The experiment therefore supports the tested sequence of schema selection, value normalization, shape control, generation, and reference-free selection on this benchmark; isolating compactness from contract guidance requires the missing factorial comparison.

7. Conclusions

MA-SQLGrid shows that a lightweight multi-stage Text-to-SQL pipeline can improve domain grounding on a fixed power-grid maintenance database. Compact schema/value context, value normalization, and answer-shape inference outperform broader baseline prompting, and reference-free validation adds a smaller final gain. Both effects, and the convention-sensitivity caveat that bounds their interpretation, replicate on a second independent generator with larger margins. The practical takeaway is simple: in a narrow operational database, better context matters more than more context, and most of the measurable benefit comes from making the expected answer contract explicit before generation.

The study is tied to one synthetic deterministic database, so its findings concern this benchmark rather than enterprise schemas generally [1,20]. Run-to-run variation is bounded only for the second generator, and hosted model changes may affect exact reproduction despite the archived traces. The second-generator C5 stage is a specification-faithful reimplementation (98.8% selection agreement), not the original binary. The row-scaling test asymmetrically stresses the full-schema condition and exposes a fixed-cap value-inventory failure. Most importantly, the present comparisons do not include the full 2×2 factorial needed to separate context scope from answer-shape hints (full versus compact context, each with and without identical shape hints). The no-shape and no-value

ablations identify channels inside C4, but a future frozen run must add a full-schema-plus-shape condition before the compactness mechanism can be generalized independently of contract guidance. The validated condition adds latency and its original C5–C4 increment is not significant. The evidence is therefore strongest for the complete contract-aware pipeline on this benchmark, not for cross-database superiority of every component.

Future work should test the same design under schema drift and with generator families beyond the two evaluated here, replace the fixed-cap value inventory with a scalable value index and re-test at larger scales, repeat the evaluation on public benchmarks, expand the error analysis by query type, and evaluate whether compact grounding transfers to other infrastructure databases with similarly structured vocabularies.

Supplementary Materials: The supplementary material contains prompt templates, dataset characterization, paired comparisons, resource-consumption results, error analyses, and the tenfold scale diagnostic.

Author Contributions: Conceptualization, L.B. and Y.Y.; methodology, L.B.; software, L.B.; validation, L.B. and C.S.; formal analysis, L.B. and C.S.; investigation, L.B. and C.S.; resources, Y.Y.; data curation, L.B. and C.S.; writing—original draft preparation, L.B.; writing—review and editing, C.S. and Y.Y.; visualization, L.B.; supervision, Y.Y.; project administration, Y.Y.; funding acquisition, Y.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Science and Technology Project of NARI Group Corporation (State Grid Electric Power Research Institute), grant number [AUTHOR INPUT REQUIRED].

Institutional Review Board Statement: Not applicable. This study did not involve humans or animals.

Informed Consent Statement: Not applicable.

Data Availability Statement: The frozen benchmark, database, code, prompts, prediction records, raw traces, evaluation outputs, and analysis scripts are available at <https://github.com/gaoxingkele/ma-sqlgrid> (release v0.2.0). The manuscript limits its claims to the artifacts available in that release and identifies the unavailable original client package and v0.1 generator script in the repository documentation.

Acknowledgments: During the preparation of this manuscript, the authors used OpenAI Codex (GPT-5-based) to assist with drafting, editing, code review, and reproducibility checks. The authors reviewed and edited the output and take full responsibility for the content of this publication. The hosted models evaluated in the experiments are research subjects; their outputs are archived in the released traces.

Conflicts of Interest: The authors declare no conflicts of interest. The funder had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

References

1. Zhong, R.; Yu, T.; Klein, D. Semantic evaluation for text-to-SQL with distilled test suites. In Proceedings of the Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP); 2020 Nov 16–20; Online, 2020. <https://doi.org/10.18653/v1/2020.emnlp-main.29>.
2. Lei, W.; Wang, W.; Ma, Z.; Gan, T.; Lu, W.; Kan, M.Y.; Chua, T.S. Re-examining the role of schema linking in text-to-SQL. In Proceedings of the Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP); 2020 Nov 16–20; Online, 2020. <https://doi.org/10.18653/v1/2020.emnlp-main.564>.
3. Liu, A.; Hu, X.; Li, L.; Wen, L. Semantic enhanced text-to-SQL parsing via iteratively learning schema linking graph. In Proceedings of the Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining; 2022 Aug 14–18; Washington, DC, USA, 2022. <https://doi.org/10.1145/3534678.3539294>.
4. Quamar, A.; Efthymiou, V.; Lei, C.; Lei, C. Natural language interfaces to data. *Found Trends Databases* **2022**, *11*, 319–414. <https://doi.org/10.1561/19000000078>.

5. Gan, Y.; Chen, X.; Huang, Q.; Purver, M.; Woodward, J.R.; Xie, J.; Huang, P. Towards robustness of text-to-SQL models against synonym substitution. In Proceedings of the Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL); 2021 Aug 1–6; Online, 2021. <https://doi.org/10.18653/v1/2021.acl-long.195>. 725
6. Liu, Q.; Yang, D.; Zhang, J.; Guo, J.; Zhou, B.; Lou, J.G. Awakening latent grounding from pretrained language models for semantic parsing. In Proceedings of the Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021; 2021 Aug 1–6; Online, 2021. <https://doi.org/10.18653/v1/2021.findings-acl.100>. 726
7. Chang, S.; Fosler-Lussier, E. How to prompt LLMs for text-to-SQL: a study in zero-shot, single-domain, and cross-domain settings, 2023. arXiv:2305.11853. 727
8. Nan, L.; Zhao, Y.; Zou, W.; Ri, N.; Tae, J.; Zhang, E.; Cohan, A.; Radev, D. Enhancing text-to-SQL capabilities of large language models: a study on prompt design strategies. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2023; 2023 Dec 6–10; Singapore, 2023. <https://doi.org/10.18653/v1/2023.findings-emnlp.996>. 728
9. Sun, R.; Arik, S.O.; Sinha, R.; Nakhost, H.; Dai, H.; Yin, P.; Pfister, T. SQLPrompt: in-context text-to-SQL with minimal labeled data, 2023. arXiv:2311.02883. 729
10. Pourreza, M.; Rafiei, D. DIN-SQL: decomposed in-context learning of text-to-SQL with self-correction, 2023. arXiv:2304.11015. 730
11. Wang, B.; Ren, C.; Yang, J.; Liang, X.; Bai, J.; Chai, L.; Zhao, Y.; Zhang, Q.; Yin, D.; Sun, X.; et al. MAC-SQL: a multi-agent collaborative framework for text-to-SQL, 2023. arXiv:2312.11242. 731
12. Talaie, S.; Pourreza, M.; Chang, Y.C.; Mirhoseini, A.; Saberi, A. CHESS: contextual harnessing for efficient SQL synthesis, 2024. arXiv:2405.16755. 732
13. Liu, X.; Shen, S.; Li, B.; Ma, P.; Runzhi, J.; Zhang, Y.; Fan, J.; Li, G.; Tang, N.; Luo, Y. A survey of text-to-SQL in the era of LLMs: where are we, and where are we going?, 2024. arXiv:2408.05109. 733
14. Katsogiannis-Meimarakis, G.; Koutrika, G. A survey on deep learning approaches for text-to-SQL. *VLDB J* **2023**, *32*, 905–936. <https://doi.org/10.1007/s00778-022-00776-8>. 734
15. Huang, X.; Ruan, W.; Huang, W.; Jin, G.; Dong, Y.; Wu, C.; Bensalem, S.; Mu, R.; Yi, Q.; Zhao, X.; et al. A survey of safety and trustworthiness of large language models through the lens of verification and validation. *Artif Intell Rev* **2024**, *57*, 175. <https://doi.org/10.1007/s10462-024-10824-0>. 735
16. Wu, T.; Terry, M.; Cai, C.J. AI chains: transparent and controllable human-AI interaction by chaining large language model prompts. In Proceedings of the Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems; 2022 Apr 29–May 5; New Orleans, LA, USA, 2022. <https://doi.org/10.1145/3491102.3517582>. 736
17. Xie, T.; Wu, C.; Shi, P.; Zhong, R.; Scholach, T.; Yasunaga, M.; Wu, C.S.; Zhong, M.; Yin, P.; Wang, S.I.; et al. UnifiedSKG: unifying and multi-tasking structured knowledge grounding with text-to-text language models. In Proceedings of the Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP); 2022 Dec 7–11; Abu Dhabi, United Arab Emirates, 2022. <https://doi.org/10.18653/v1/2022.emnlp-main.39>. 737
18. Li, Z.; Xie, T. Using LLM to select the right SQL query from candidates, 2024. arXiv:2401.02115. 738
19. Lee, C.H.; Polozov, O.; Richardson, M. KaggleDBQA: realistic evaluation of text-to-SQL parsers. In Proceedings of the Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL); 2021 Aug 1–6; Online, 2021. <https://doi.org/10.18653/v1/2021.acl-long.176>. 739
20. Lei, F.; Chen, J.; Ye, Y.; Cao, R.; Shin, D.; Su, H.; Suo, Z.; Gao, H.; Hu, W.; Yin, P.; et al. Spider 2.0: evaluating language models on real-world enterprise text-to-SQL workflows, 2024. arXiv:2411.07763. 740
21. Herzig, J.; Nowak, P.K.; Muller, T.; Piccinno, F.; Eisenschlos, J.M. TaPas: weakly supervised table parsing via pre-training. In Proceedings of the Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL); 2020 Jul 5–10; Online, 2020. <https://doi.org/10.18653/v1/2020.acl-main.398>. 741
22. Chen, W. Large language models are few(1)-shot table reasoners. In Proceedings of the Findings of the Association for Computational Linguistics: EACL 2023; 2023 May 2–6; Dubrovnik, Croatia, 2023. <https://doi.org/10.18653/v1/2023.findings-eacl.83>. 742
23. Zhang, Q.; Chen, H.; Dong, J.; Chen, S.; Huang, F.; Huang, X. Structure guided large language model for SQL generation, 2024. arXiv:2402.13284. 743
24. Liu, X.; Tan, Z. Divide and prompt: chain of thought prompting for text-to-SQL, 2023. arXiv:2304.11556. 744
25. Arora, A.; Bhaisaheb, S.; Nigam, H.; Patwardhan, M.; Vig, L.; Shroff, G. Adapt and decompose: efficient generalization of text-to-SQL via domain adapted least-to-most prompting, 2023. arXiv:2308.02582. 745
26. Xie, Y.; Jin, X.; Xie, T.; Matrixmxlin, M.; Chen, L.; Yu, C.; Lei, C.; Zhuo, C.; Hu, B.; Zang, L. Decomposition for enhancing attention: improving LLM-based text-to-SQL through workflow paradigm. In Proceedings of the Findings of the Association for Computational Linguistics: ACL 2024; 2024 Aug 11–16; Bangkok, Thailand, 2024. <https://doi.org/10.18653/v1/2024.findings-acl.641>. 746
27. Xie, W.; Wu, G.; Zhou, B. MAG-SQL: multi-agent generative approach with soft schema linking and iterative sub-SQL refinement for text-to-SQL, 2024. arXiv:2408.07930. 747

28. Cen, J.; Liu, J.; Li, Z.; Wang, J. SQLFixAgent: towards semantic-accurate text-to-SQL parsing via consistency-enhanced multi-agent collaboration. In Proceedings of the Proceedings of the 39th AAAI Conference on Artificial Intelligence; 2025 Feb 25–Mar 4; Philadelphia, PA, USA, 2025. <https://doi.org/10.1609/aaai.v39i1.31979>.
29. Guo, Z.; Jin, R.; LIU, C.; Huang, Y.; Shi, D.; Supryadi.; Yu, L.; Liu, Y.; Li, J.; Xiong, B.; et al. Evaluating large language models: a comprehensive survey, 2023. arXiv:2310.19736.
30. Wang, B.; Lapata, M.; Titov, I. Learning from executions for semantic parsing. In Proceedings of the Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT); 2021 Jun 6–11; Online, 2021. <https://doi.org/10.18653/v1/2021.naacl-main.219>.
31. Chen, Z.; Chen, S.; White, M.; Mooney, R.J.; Payani, A.; Srinivasa, J.; Su, Y.; Sun, H. Text-to-SQL error correction with language models of code. In Proceedings of the Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Short Papers); 2023 Jul 9–14; Toronto, ON, Canada, 2023. <https://doi.org/10.18653/v1/2023.acl-short.117>.
32. Beyer, D.; Dangl, M.; Dietsch, D.; Heizmann, M.; Lemberger, T.R.; Tautschnig, M. Verification witnesses. *ACM Trans Softw Eng Methodol* **2022**, *31*, 1–69. <https://doi.org/10.1145/3477579>.
33. Pauwels, P.; van den Bersselaar, E.; Verhelst, L. Validation of technical requirements for a BIM model using semantic web technologies. *Adv Eng Inform* **2024**, *60*, 102426. <https://doi.org/10.1016/j.aei.2024.102426>.
34. Zhai, B.; Xu, C.; He, Y.; Yao, Z. ExCoT: optimizing reasoning for text-to-SQL with execution feedback, 2025. arXiv:2503.19988.
35. Pourreza, M.; Li, H.; Sun, R.; Chung, Y.; Talaei, S.; Kakkar, G.T.; Gan, Y.; Saber, A.; Ozcan, F.; Ark, S.O. CHASE-SQL: multi-path reasoning and preference optimized candidate selection in text-to-SQL, 2024. arXiv:2410.01943.
36. Soliman, H.; Gupta, V.; Roth, D.; Gurevych, I. CORE-T: COherent RETrieval of tables for text-to-SQL, 2026. arXiv:2601.13111.
37. Ganesan, P.; Jha, R.A.; Roth, D.; Gupta, V. UNJOIN: enhancing multi-table text-to-SQL generation via schema simplification, 2025. arXiv:2505.18122.
38. Yang, H.; Zhang, J.; He, Z.; Zhou, A.; Fung, Y.R. MARS-SQL: a multi-agent reinforcement learning framework for text-to-SQL, 2025. arXiv:2511.01008.
39. Li, C.; Shao, Y.; Li, Y.; Liu, Z. SEA-SQL: semantic-enhanced text-to-SQL with adaptive refinement, 2024. arXiv:2408.04919.
40. Luo, L.; Xie, H.; Shen, S.; Ma, Z.; Ling, R.; Xu, H.; Jiang, H.; Chen, D.; Li, Y.; Chen, P.; et al. SIRIUS-SQL: anchoring multi-candidate text-to-SQL in execution feedback, 2026. arXiv:2606.01246.
41. Tremante, A.; He, Y.; Klopfenstein, R.; Wang, Y.; Narodytska, N.; Wu, H. SpotIt+: verification-based text-to-SQL evaluation with database constraints, 2026. arXiv:2603.04334.
42. Zhou, Y.; Zhang, F.; Guo, Z.; Chen, Y.; Zhang, H.; Nakov, P.; Xie, Z. SQLStructEval: structural evaluation of LLM text-to-SQL generation, 2026. arXiv:2604.06736.
43. Arif, T.J.; Singh, K. Agent-agnostic evaluation of SQL accuracy in production text-to-SQL systems, 2026. arXiv:2604.28049.
44. Borovčák, K.; Bađić Babac, M.; Mornar, V. Evaluating open-source LLM agents for SQL generation and structured analytics on relational databases. *Comput Mater Contin* **2026**, *88*, 80. <https://doi.org/10.32604/cmc.2026.078330>.
45. Lin, X.; Socher, R.; Xiong, C. Bridging textual and tabular data for cross-domain text-to-SQL semantic parsing. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2020; 2020 Nov 16–20; Online, 2020. <https://doi.org/10.18653/v1/2020.findings-emnlp.438>.
46. Brunner, U.; Stockinger, K. ValueNet: a natural language-to-SQL system that learns from database information, 2020. arXiv:2006.00888.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.